Sistema para la gestión del servicio de alimentación

Trabajo integrador Cursada 2026

FECHA DE ENTREGA 16/10/2026

Destinatario: Hospital Interzonal de Agudos Especializado en Pediatría "Sor María Ludovica". Servicio de Alimentación y Dietética

Contexto

El Servicio de Alimentación y Dietética del Hospital Interzonal de Agudos Especializado en Pediatría **“Sor María Ludovica”** tiene a su cargo la gestión de las comidas destinadas al personal autorizado de los diferentes sectores/ servicios del hospital.

Para organizar esta actividad es necesario coordinar tres instancias de un mismo circuito:

- La declaración de las personas o cantidades de viandas requeridas por cada servicio/sector.
- La revisión y autorización de esos pedidos por parte del Servicio de Alimentación.
- La entrega física de las comidas y el registro de quién las retira.

Actualmente, estas actividades se realizan mediante planillas en papel, listados impresos, comunicaciones telefónicas y controles manuales. Cada sector informa sus necesidades, el Servicio de Alimentación revisa y ajusta las cantidades solicitadas, y el personal del comedor verifica posteriormente las autorizaciones al momento de efectuar la entrega.

La falta de integración entre estas etapas provoca diferentes dificultades operativas y administrativas:

- Pedidos duplicados o superiores a las necesidades reales.
- Inconsistencias entre las cantidades solicitadas, autorizadas y entregadas.

- Cambios de personal o de guardias que no son comunicados oportunamente.
- Necesidad de realizar verificaciones y llamados entre los distintos sectores.
- Falta de trazabilidad sobre quién solicitó, autorizó y retiró cada comida.
- Dificultades para controlar los diferentes tipos de menú.
- Escasa disponibilidad de información consolidada para auditoría, planificación y control del gasto.
- Desperdicio de alimentos como consecuencia de sobrepedidos o entregas no registradas.

Deberá permitir gestionar de manera independiente el **desayuno**, **almuerzo, merienda** y la **cena**, así como distinguir diferentes tipos de menú, a saber:

- General.
- Vegetariano.
- Celíaco.

Para resolver esta problemática se propone desarrollar una aplicación web que digitalice el circuito completo de solicitud, autorización y entrega de comidas. El sistema será utilizado por tres actores principales:

- Los **Jefes de Servicio**, responsables de declarar las necesidades de alimentación de su sector.
- El **Servicio de Alimentación**, encargado de revisar, ajustar y autorizar los pedidos.
- El **Sector Comedor**, responsable de verificar las autorizaciones y entregar las comidas.

Cada actor dispondrá de una vista específica dentro del sistema, de acuerdo con sus responsabilidades y permisos. Sin embargo, todos trabajarán sobre información integrada: los pedidos cargados por los Jefes de Servicio serán revisados por el Servicio de Alimentación, y únicamente las cantidades autorizadas estarán disponibles para su entrega en el comedor.

La solución deberá mantener diferenciadas las cantidades **pedidas**, **autorizadas** y **entregadas**, registrar el historial de las operaciones realizadas y permitir conocer en todo momento el estado de cada solicitud. De esta manera, se buscará mejorar la coordinación entre las áreas, reducir errores y desperdicios, y garantizar la trazabilidad integral del proceso.

Funcionalidades de la aplicación (se dividirá en dos etapas)

El desarrollo se organizará en dos etapas. La primera comprenderá la aplicación privada de administración, el modelo de datos y la API del sistema. La segunda comprenderá la aplicación web utilizada por los tres roles operativos —Jefe de Servicio, Servicio de Alimentación y Sector Comedor— e implementará el circuito de solicitud, autorización y entrega de comidas.

A continuación se describen, en líneas generales, las principales funcionalidades esperadas para cada actor.

Jefe de Servicio

El Jefe de Servicio será responsable de registrar y consultar los pedidos de comidas correspondientes a su sector.

El sistema deberá permitirle:

- Consultar e incorporar el personal asociado a su sector y eventuales personas de tipo Residente a los pedidos de su sector.
- Registrar pedidos nominados de comidas, identificando a cada persona mediante nombre, apellido y DNI.
- Gestionar de manera independiente las solicitudes de desayuno, almuerzo, merienda y cena.
- Utilizar automáticamente el tipo de menú asociado a cada persona.
- Realizar cargas con periodicidad diaria, semanal o mensual.
- Visualizar los cupos correspondientes a su sector para cada tipo de comida.
- Identificar las solicitudes que quedan comprendidas dentro del cupo Regular y aquellas que correspondan a Excepción.
- Consultar y modificar los pedidos de su sector mientras éstos no hayan sido autorizados o rechazados por el Servicio de Alimentación.
- Consultar pedidos actuales e históricos.

Cuando un sector tenga más de un Jefe de Servicio, todos ellos podrán consultar y gestionar los pedidos correspondientes a dicho sector.

Servicio de Alimentación

El Servicio de Alimentación será responsable de revisar los pedidos realizados por los distintos sectores y determinar qué solicitudes quedan autorizadas.

El sistema deberá permitirle:

- Consultar los pedidos realizados por todos los sectores.
- Visualizar las solicitudes correspondientes a desayuno, almuerzo, merienda y cena de manera independiente.
- Identificar claramente las solicitudes clasificadas como Regulares y aquellas realizadas por Excepción.
- Autorizar o no autorizar de manera nominada las solicitudes realizadas.
- Realizar autorizaciones individuales o masivas para agilizar la operatoria.
- Rechazar completamente un pedido cuando corresponda.
- Mantener diferenciada la información originalmente solicitada de la posteriormente autorizada.
- Administrar los cupos correspondientes a cada sector y tipo de comida.
- Consultar pedidos y autorizaciones actuales e históricas.
- Obtener totales y resúmenes por sector, fecha, tipo de comida y tipo de menú.
- Exportar listados y consultar información consolidada para control y planificación.

Las cantidades autorizadas deberán obtenerse a partir de las solicitudes nominadas efectivamente autorizadas y no deberán sobrescribir la información del pedido original.

Sector Comedor

El Sector Comedor será responsable de consultar las comidas autorizadas y registrar las entregas realizadas.

El sistema deberá permitirle:

- Buscar personas mediante DNI, nombre o apellido.
- Consultar las comidas autorizadas para cada persona en una fecha determinada.
- Identificar de manera independiente si una persona tiene autorizado desayuno, almuerzo, merienda o cena.
- Visualizar el tipo de menú correspondiente.
- Verificar si una comida autorizada ya fue entregada.
- Registrar la entrega de una o varias comidas en una misma operación.

- Permitir que una persona retire comidas correspondientes a otras personas autorizadas.
- Registrar quién fue el responsable efectivo del retiro.
- Impedir que una misma comida autorizada sea entregada más de una vez.
- Consultar las entregas realizadas durante el día.
- Visualizar totales autorizados y entregados.

Requerimientos funcionales generales

Independientemente del actor que utilice cada funcionalidad, el sistema deberá:

- Gestionar usuarios, roles y permisos de acceso.
- Mantener un registro de los sectores o servicios del hospital y sus responsables.
- Mantener un registro del personal que podrá ser incluido en los pedidos de comidas.
- Permitir la carga manual y la importación masiva de personal.
- Contemplar los distintos tipos de personal y tipos de menú definidos para el sistema.
- Mantener todos los pedidos de comida de forma nominada.
- Gestionar desayuno, almuerzo, merienda y cena de manera independiente.
- Mantener diferenciadas las solicitudes realizadas, las autorizaciones otorgadas y las entregas efectivamente registradas.
- Mantener la condición Regular o Excepción asignada a cada solicitud durante el posterior circuito de autorización y entrega.
- Registrar fecha, hora y usuario responsable de las operaciones relevantes.
- Conservar la información histórica necesaria para garantizar la trazabilidad del circuito.
- Proporcionar resúmenes, estadísticas e historiales por período, sector, tipo de comida y tipo de menú.
- Implementar controles de acceso y validaciones tanto en la interfaz como en el backend.
- Proporcionar interfaces adaptadas a las funciones y permisos correspondientes a cada tipo de usuario.

La distribución detallada de estas funcionalidades entre la primera y la segunda etapa se especificará posteriormente.

Componentes de la aplicación

- **Backend en Python y Flask:** implementará la lógica de negocio, las validaciones del lado del servidor, la autenticación, el control de roles y permisos, y el acceso a la información almacenada. También será responsable de aplicar las reglas correspondientes a la carga, autorización y entrega de comidas.
- **Base de datos PostgreSQL:** almacenará la información de usuarios, roles, permisos, sectores, personal, solicitudes, detalles de pedidos, cupos, autorizaciones, entregas e historial de operaciones. Para la interacción con la base de datos se utilizará SQLAlchemy como ORM.
- **API para consultas e integración:** el backend desarrollado en Flask expondrá los endpoints necesarios para que la aplicación frontend pueda consultar y modificar la información del sistema. La comunicación se realizará mediante solicitudes HTTP y el intercambio de datos en formato JSON.
- **Aplicación frontend en Vue.js 3:** se desarrollará una aplicación web que consumirá la API implementada en Flask. La aplicación contará con diferentes vistas y funcionalidades según el rol del usuario autenticado:
    - Jefe de Servicio.
    - Servicio de Alimentación.
    - Sector Comedor.
- **Interfaz responsive y mobile first:** las pantallas deberán adaptarse a computadoras de escritorio, notebooks, tablets y teléfonos móviles. La vista utilizada por el Sector Comedor deberá estar especialmente optimizada para tablets y ofrecer una modalidad de interacción tipo kiosco, con controles de tamaño adecuado y teclado numérico en pantalla para realizar búsquedas por DNI.

Etapa 1

ENTREGA 16/10/2026

Alcance : Aplicación Privada y API

Durante la primera etapa se desarrollará la **aplicación privada de administración**, el modelo de datos y la **API** necesaria para implementar el circuito completo de solicitud, autorización y entrega de comidas.

La aplicación privada estará destinada a usuarios con rol **Administrador**. Permitirá gestionar la información estructural y de configuración utilizada por las diferentes áreas del sistema, incluyendo usuarios, roles, sectores, personal, tipos de menú y otros parámetros generales.

El Administrador tendrá acceso completo a los módulos de la aplicación privada y será responsable de:

- Gestionar usuarios.
- Asignar roles a los usuarios.
- Activar o bloquear cuentas.
- Gestionar sectores o servicios.
- Asignar responsables a los sectores.
- Gestionar el personal del hospital.
- Administrar tipos de personal.
- Administrar tipos de menú de comidas.
- Configurar otros parámetros generales requeridos por el sistema.

La aplicación se desarrollará utilizando **Python y Flask**, con **Jinja** como motor de plantillas, **PostgreSQL** como base de datos y **SQLAlchemy** como ORM.

Además de la aplicación privada, durante esta etapa se implementará una **API** que exponga las operaciones necesarias para las interfaces operativas que se desarrollarán en Vue.js durante la segunda etapa.

Roles del sistema

Quedarían definidos cuatro roles:

- **Administrador:** gestiona la configuración general, los usuarios, sectores, personal y parámetros del sistema.
- **Jefe de Servicio:** registra y consulta los pedidos correspondientes al personal de su sector.
- **Servicio de Alimentación:** revisa, ajusta, autoriza o rechaza pedidos y administra los cupos.

- **Sector Comedor:** consulta las autorizaciones vigentes y registra las entregas realizadas.

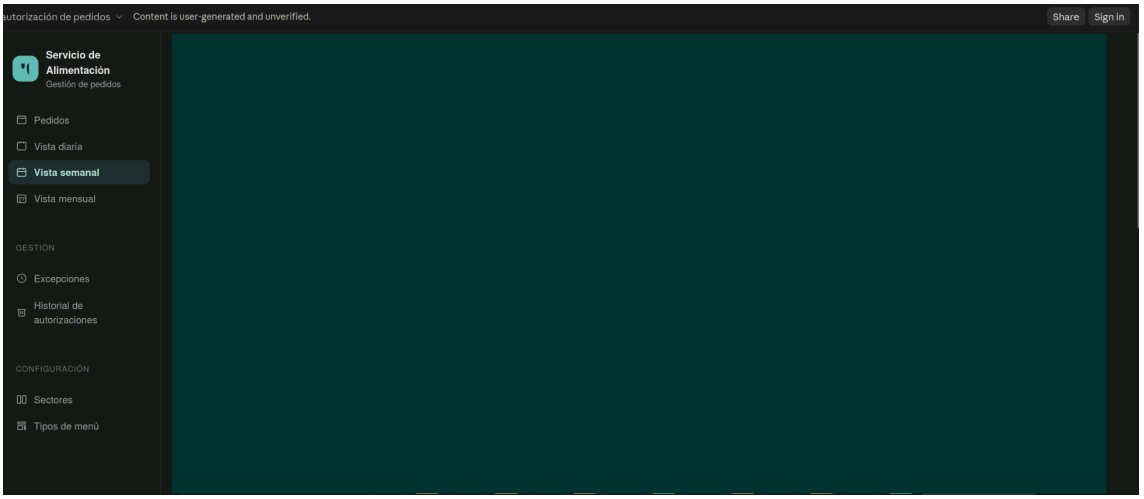
El Administrador podrá crear usuarios y asignar cualquiera de los tres roles operativos. También podrá crear otros usuarios administradores.

1. Layout

Descripción

Implementar un **layout base** para todas las vistas de la aplicación privada de administración. Este layout funcionará como contenedor principal y deberá garantizar una estructura visual uniforme, permitiendo que cada módulo reemplace únicamente el contenido central sin modificar los elementos generales de navegación.

El layout deberá implementarse mediante una plantilla base de **Jinja**, que será extendida por las diferentes vistas de la aplicación Flask. De esta manera, los elementos comunes —encabezado, navegación, mensajes y datos de la sesión— deberán definirse una sola vez.



* El layout es solo un ejemplo, carece de opciones relevantes. Cada grupo es libre de la implementación de su interfaz.

Requerimientos funcionales

- Implementar un menú de navegación lateral o superior con enlaces a los módulos de la aplicación privada:

    - Gestión de usuarios.
    - Gestión de sectores o servicios.
    - Gestión de personal.
    - Gestión de tipos de menú.
    - Parámetros generales del sistema, en caso de incorporarse.
- El menú deberá indicar visualmente cuál es el módulo o sección activa.
- Implementar una barra superior que incluya:
    - Nombre o logo de la aplicación.
    - Nombre y apellido del usuario autenticado.
    - Menú de cuenta con las opciones:
        - Ver perfil.
        - Cerrar sesión.
- Incorporar un área central destinada al contenido particular de cada módulo.
- Incorporar un mecanismo común para mostrar mensajes de información, confirmación, advertencia o error generados por Flask.

2. Módulo de usuarios

Descripción

Implementar el módulo de usuarios de la aplicación privada, accesible únicamente para usuarios con rol **Administrador**. Este módulo permitirá crear, consultar, actualizar y eliminar usuarios del sistema, asignarles un rol y, cuando corresponda, asociarlos a un sector del hospital.

Requerimientos funcionales

- Operaciones CRUD de usuarios.
- Validar que no existan dos usuarios con el mismo correo electrónico.
- Campos mínimos para cada usuario:
    - Correo electrónico.
    - Nombre.
    - Apellido.
    - DNI
    - Contraseña.
    - Activo: Sí | No.
    - Rol:
        - Administrador.

        - Jefe de Servicio.
        - Servicio de Alimentación.
        - Sector Comedor.
    - ~~Sector, obligatorio únicamente para usuarios con rol Jefe de Servicio.~~
    - Fecha de creación.
    - Fecha de última modificación.
- Reglas para la asignación de sectores:
    - ~~Un usuario con rol Jefe de Servicio deberá estar asociado a un único sector.~~
    - Un sector podrá tener uno o más usuarios con rol Jefe de Servicio.
    - Todos los Jefes asociados a un mismo sector podrán cargar y consultar pedidos correspondientes a dicho sector.
    - Los usuarios con rol Administrador, Servicio de Alimentación o Sector Comedor no tendrán un sector asociado.
    - Si se modifica el rol de un Jefe de Servicio, deberá eliminarse su asociación con el sector.
    - No se podrá asociar un usuario a un sector inactivo.
    - No se podrá desactivar un sector con usuarios activos.
- Funcionalidad de búsqueda por:
    - Correo electrónico.
    - Nombre o apellido.
    - DNI
    - Activo: Sí | No.
    - Rol.
    - Sector o servicio.
- Los filtros deberán poder combinarse entre sí.
- Resultados ordenables por:
    - Fecha de creación.
    - Apellido y nombre.
    - Correo electrónico.
- El ordenamiento deberá permitir sentido ascendente y descendente.
- Paginación del lado del servidor, con un máximo de **25 registros por página**.
- En la edición de un usuario:
    - La contraseña no será obligatoria.
    - Si el campo de contraseña queda vacío, deberá conservarse la contraseña actual.

    - Si se ingresa una nueva contraseña, deberá almacenarse utilizando un hash seguro.
- No se podrá eliminar un usuario que tenga pedidos, autorizaciones, entregas u otras operaciones asociadas. En esos casos deberá desactivarse mediante la funcionalidad de bloqueo definida en el módulo siguiente.

Notas

- Validar en el cliente:
    - Campos requeridos.
    - Formato de correo electrónico.
    - Formato y longitud de nombre y apellido.
    - Longitud mínima y confirmación de contraseña.
    - Selección obligatoria de un sector cuando el rol sea Jefe de Servicio.
- Validar en el servidor:
    - Campos requeridos.
    - Formato y longitud de los campos.
    - Unicidad del correo electrónico.
- Las contraseñas deberán almacenarse mediante un algoritmo seguro de hash y nunca podrán mostrarse en formularios, listados, registros de log o respuestas de la API.
- Se deberá mostrar un feedback claro ante errores de validación y ante el resultado de las operaciones realizadas.

Permisos

- Solamente los usuarios con rol **Administrador** podrán acceder a este módulo.

3. Gestión de Roles, Permisos y Bloqueo de Usuarios

Descripción

Implementar las funcionalidades de asignación de roles, control de permisos y bloqueo o desbloqueo de usuarios.

Cada usuario tendrá asignado **un único rol**, que determinará las funcionalidades a las que puede acceder. Los permisos deberán utilizarse para

controlar las acciones específicas que cada rol puede realizar, tanto en la aplicación privada desarrollada con Flask y Jinja como en los endpoints de la API.

Los roles definidos para el sistema serán:

- Administrador.
- Jefe de Servicio.
- Servicio de Alimentación.
- Sector Comedor.

Requerimientos funcionales

- Asignar o desasignar roles a un usuario (Administrador).
- Bloquear/desbloquear usuarios:
  - Un usuario bloqueado no puede iniciar sesión.
  - Los usuarios con rol Administrador no pueden ser bloqueados.
- Implementar permisos siguiendo el patrón modulo_accion. Ejemplos para este módulo:
  - user_index → listar usuarios
  - user_new → crear usuario
  - user_update → actualizar usuario
  - user_destroy → eliminar usuario
  - user_show → ver detalle de usuario

Notas

- Los permisos y roles pueden administrarse directamente desde la base de datos en esta etapa del proyecto. Es decir, no hace falta hacer una interfaz gráfica para agregar permisos y roles, pero deberán existir en la base de datos.
- **Es importante entender el porqué del uso de esta solución para implementar la autorización y seguir el esquema de forma correcta. Este tema será explicado oportunamente en los horarios de práctica.**

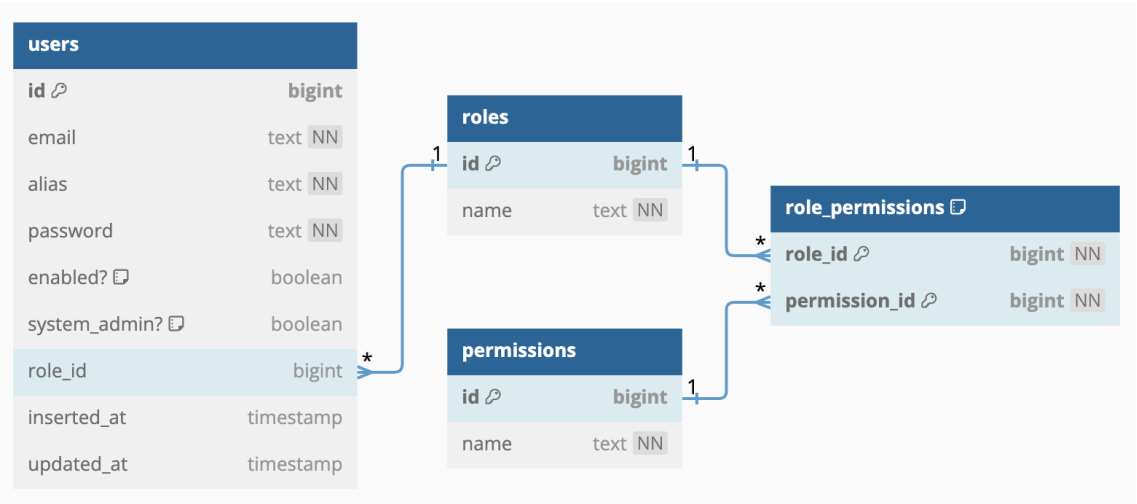


Figura 2. Posible esquema para el manejo de usuarios.

Permisos

- Todas las operaciones de roles y permisos sólo pueden ser realizadas por un usuario con rol **Administrador**.

4. Login y manejo de sesiones

Descripción

Implementar el sistema de autenticación y manejo de sesiones para la aplicación privada (Administración).

Requerimientos funcionales

- Formulario de login para la aplicación privada que permita iniciar sesión con email y contraseña.
- Validación de credenciales contra la base de datos.
- Manejo seguro de contraseñas (hash).
- Creación y manejo de sesiones para mantener el estado del usuario autenticado.
- Verificación de sesión y permisos en cada módulo.
- Implementar cierre de sesión.

Notas

- **No se permite el uso de librerías externas que abstraigan el proceso completo de login (por ejemplo, flask-login)**. La implementación debe ser

manual para comprender y controlar cada paso del flujo de autenticación.
- Implementar verificación de permisos antes de acceder a funcionalidades restringidas. Esta verificación se debe realizar para todas las secciones del sistema de Administración.
- Las cookies de sesión deben ser seguras y tener fecha de expiración.
- **Deben utilizar** la librería **Flask-Session** para almacenar las sesiones del lado del servidor.
- La autenticación en la aplicación privada será obligatoria para acceder cualquier parte de la aplicación de administración.

5. Gestión de Sectores

Descripción

Implementar el módulo de sectores del hospital. Este módulo permitirá crear, consultar, actualizar y eliminar los sectores a los que pertenecen el personal y los Jefes de Servicio.

Requerimientos funcionales

- Operaciones CRUD de sectores.
- Validar que no existan dos sectores con el mismo nombre.
- Campos mínimos para cada sector:
  - Nombre.
  - Descripción.
  - Activo: Sí | No.
  - Fecha de creación.
  - Fecha de última modificación.
- Permitir asociar uno o más usuarios con rol **Jefe de Servicio** a un sector.
- Cada Jefe de Servicio podrá estar asociado a un único sector.
- Un sector podrá tener varios Jefes de Servicio.
- Los Jefes de Servicio asociados a un mismo sector podrán cargar y consultar pedidos de ese sector.
- Funcionalidad de búsqueda por:
  - Nombre.
  - Estado: Activo | Inactivo.
  - Jefe de Servicio asignado.

- Resultados ordenables por:
  - Nombre.
  - Fecha de creación.
- El ordenamiento deberá permitir sentido ascendente y descendente.
- Paginación del lado del servidor, con un máximo de **25 registros por página**.

Notas

- Validar en el cliente:
  - Campos requeridos.
  - Formato y longitud de los campos.
- Validar en el servidor:
  - Campos requeridos.
  - Unicidad del nombre.
  - Existencia de los usuarios seleccionados.
  - Que los usuarios asignados tengan rol Jefe de Servicio.
  - Que un Jefe de Servicio no se encuentre asociado a otro sector.
- No se podrá eliminar un sector que tenga personal, usuarios o pedidos asociados. En esos casos deberá desactivarse.
- Un sector inactivo no podrá utilizarse para nuevas asignaciones de personal ni para nuevos pedidos.
- La desactivación de un sector no deberá eliminar sus registros históricos.

Permisos

- Solamente los usuarios con rol **Administrador** podrán acceder a este módulo.

6. Gestión del Personal del Hospital

Descripción

Implementar el módulo de personal del hospital. Este módulo permitirá registrar, consultar, actualizar e importar las personas que podrán ser incluidas en los pedidos nominados de comidas.

Importante : El personal registrado no necesariamente tendrá una cuenta para acceder al sistema. Los **usuarios** representan a quienes utilizan la aplicación,

mientras que el **personal** representa a las personas para las que se solicitan y entregan comidas. Una excepción es el caso de los Jefes de Servicio que son usuarios registrados y a la vez pueden ser personal autorizado para las comidas.

El sistema deberá contemplar que algunas personas, particularmente los **residentes**, pueden rotar entre distintos sectores del hospital por lo tanto no tendrán sector asignado y podrá incorporarse a pedidos de cualquier sector.

Requerimientos funcionales

- Operaciones CRUD de personal.
- Campos mínimos para cada persona:
  - DNI.
  - Nombre.
  - Apellido.
  - Tipo de personal. [*1]
  - Sector (No aplica para residentes) [*2]
  - Menu[*3]
  - Activo: Sí | No.
  - Fecha de creación.
  - Fecha de última modificación.
- Validar que no existan dos personas con el mismo DNI .
- El personal fijo está asociado a un único sector a la vez. Para cambiar de sector alcanza con reemplazar ese valor;
- El personal de tipo Residente no tiene sector asignado obligatoriamente y puede incorporarse al pedido nominado de cualquier sector
- Para incorporar una persona de tipo Personal fijo a un pedido, su sector debe coincidir con el sector del Jefe de Servicio que realiza la carga. Los Residentes pueden incorporarse a un pedido de cualquier sector sin esa validación
- Permitir la búsqueda por:
  - DNI.
  - Nombre o apellido.
  - Sector.
  - Tipo de personal.
  - Estado.
- Resultados ordenables por:
  - Apellido y nombre.

    - DNI.
    - Fecha de creación.
- Paginación del lado del servidor, con un máximo de 25 registros por página.
- Permitir la importación masiva de personal mediante un archivo CSV. Para esta funcionalidad se espera que se defina un formato de CSV, luego el usuario administrador cargue el personal en él y cuando se importe desde el sistema se añadan el personal nominado en él. Es importante validar cada registro/persona e informar el error en caso de suceder.

Notas

- Validar en el cliente y en el servidor:
    - Campos requeridos.
    - Formato de DNI.
    - Unicidad de DNI
    - Existencia del tipo de personal.
    - Existencia de los sectores asignados.
- No se podrá eliminar una persona que tenga pedidos o entregas asociados. En esos casos deberá marcarse como inactiva.
- Una persona inactiva no podrá incorporarse a nuevos pedidos, pero deberá mantenerse visible en los registros históricos.
- El cambio de sector de una persona no deberá modificar pedidos, autorizaciones o entregas registrados anteriormente.
- Cada persona tendrá asociado un tipo de menú predeterminado en su ficha de Personal. Al incorporarla a un pedido, el sistema utilizará automáticamente dicho tipo de menú y lo almacenará en el detalle del pedido, sin necesidad que el Jefe de servicio deba asignarlo manualmente. Posteriores cambios en la ficha de la persona no deben modificar pedidos históricos.
- La importación masiva deberá informar los registros procesados, importados y rechazados, indicando el motivo de cada error.

Permisos

- Solamente los usuarios con rol **Administrador, Jefe de Servicio y Servicio de Alimentación** podrán gestionar el personal y sus asignaciones desde la aplicación.

[*1] Sobre el Tipo de Personal:

Tipos de personal

Los tipos de personal serán datos predefinidos, por ejemplo:

- Residente.
- Personal Fijo.

Deberán almacenarse obligatoriamente en una tabla independiente de la base de datos y relacionarse con el personal mediante una clave foránea, garantizando la integridad referencial. Los valores podrán cargarse mediante migraciones, scripts de inicialización o datos precargados. Opcionalmente, el grupo podrá implementar un CRUD en la aplicación privada para que el Administrador gestione los tipos de personal.

[*2] **Sobre los Sectores/Servicios**

Los sectores/servicios existentes son los siguientes. Instanciar a modo de ejemplo alguno o la totalidad de los mismos. A saber:

Medicos Internos, Medicos De Guardia (Urg)., Quirofano , Equipo De Guardia , Intendencia , Policia , Mantenimiento , Electricidad, Conmutador, Admision De Camas, Prep De Materiales, Laboratorio De Guardia, Economia Y Gestion , Hemoterapia , Secretarias De Guardia Estadistica, Rayos, Esterilizacion, Ssp, Mensajeria , Ate, Limpieza , Cefli , Lavadero, Salud Mental , Quirofano Cardiovascular , Alimentacion, Direccion, Deposito Central, Alegria Intensiva, Estadistica, Neurocirugia, Hermanas, Residentes Jefes, Niño Sano 63 Y 16, Genetica Idip

[*3] Sobre el Tipo de Menu:

Tipos de menú

Los tipos de menú serán datos predefinidos:

- General.
- Vegetariano.
- Celiaco.

Deberán almacenarse obligatoriamente en una tabla independiente de la base de datos y relacionarse con el Personal y, según el modelo adoptado, con la información histórica correspondiente a las solicitudes de comidas. Los valores podrán cargarse mediante migraciones, scripts de inicialización o datos precargados. Opcionalmente, el grupo podrá implementar un CRUD en la aplicación privada para que el Administrador gestione los tipos de menú.

7. Cupos

Descripción

Implementar el módulo de cupos por sector. Este módulo permitirá definir, para cada sector y tipo de comida (desayuno, almuerzo, merienda y cena), la cantidad máxima de personas que podrán incorporarse como solicitudes regulares.

Todos los pedidos de comidas serán nominados, por lo que cada comida solicitada deberá estar asociada a una persona identificada.

El cupo no representa una modalidad diferente de pedido, sino que se utilizará como límite para determinar si cada persona incorporada a un pedido corresponde a una **solicitud regular o a una solicitud por excepción**.

Requerimientos funcionales

- Operaciones CRUD de cupos.
- Campos mínimos para cada cupo:
    - Sector.
    - Tipo de comida:
        - Desayuno.
        - Almuerzo.
        - Merienda.
        - Cena.
    - Cantidad de comensales.
    - Fecha de creación.
    - Fecha de última modificación.
- El cupo será independiente del tipo de menú.
- Para cada fecha y tipo de comida, las personas incorporadas al pedido hasta alcanzar el cupo disponible serán consideradas **regulares**.
- Las personas incorporadas una vez alcanzado el cupo serán consideradas **excepciones**.

- El sistema deberá identificar automáticamente esta condición durante la carga del pedido.
- No se podrá crear un cupo para un sector inactivo.
- No podrán coexistir dos cupos vigentes para el mismo sector y el mismo tipo de comida
- Funcionalidad de búsqueda por:
    - Sector.
    - Tipo de comida.
- Resultados ordenables por:
    - Sector.

Paginación del lado del servidor, con un máximo de 25 registros por página.

Notas

- Validar en el cliente y en el servidor:
    - Campos requeridos.
    - Que la cantidad sea mayor que cero.
    - Existencia y estado activo del sector.
- La modificación de un cupo no deberá alterar retroactivamente pedidos que ya se encuentren confirmados, autorizados o entregados.
- El tipo de menú no modifica el consumo del cupo. Cada persona ocupa una unidad del cupo independientemente de que corresponda menú General, Vegetariano o Celíaco.

Permisos

Los cupos serán administrados por usuarios con rol **Servicio de Alimentación** mediante la API. El Jefe de Servicio podrá consultar los correspondientes a su sector, pero no modificarlos.

Sobre los pedidos de comida

Los pedidos deberán permitir representar de manera independiente las solicitudes correspondientes a **desayuno, almuerzo, merienda y cena** para cada **sector** y **fecha**. Todas las solicitudes serán nominadas, por lo que cada comida solicitada deberá quedar asociada a una persona identificada.

Para cada solicitud deberá poder determinarse, como mínimo, el sector, la fecha, el tipo de comida, la persona, el tipo de menú y su condición **Regular** o **Excepción**.

La organización interna de esta información y el modelo de datos utilizado quedarán a criterio de cada grupo, siempre que permitan cumplir las reglas de negocio y mantener la trazabilidad requerida.

La estructura de la API podrá adaptarse al modelo de datos y a las decisiones de diseño adoptadas por cada grupo. Recuerden que los endpoints propuestos en este documento deberán considerarse como una guía y podrán ajustarse, agruparse o complementarse cuando resulte necesario, siempre que se mantengan las funcionalidades, reglas de negocio y restricciones de acceso definidas.

8. Ajustes del sistema

Este módulo permitirá administrar la configuración del sistema, como mínimo deberá contemplar la siguiente configuración:

- Cantidad de elementos por página en los listados del sistema (todos los listados deberán respetar este valor para el paginado).
- Posibilidad de deshabilitar el sitio para mantenimiento
- Mensaje de mantenimiento de sitio

La configuración del sistema sólo podrá modificarla un usuario con el rol de Administrador.

9. API

Descripción

Implementar una API que permita acceder a las funcionalidades necesarias para las tres interfaces que se desarrollarán en Vue.js durante la segunda etapa:

- Jefe de Servicio.
- Servicio de Alimentación.
- Sector Comedor.

NOTAS:

- La siguiente lista es una guía pero que el/la estudiante pueda modificar lo definido o agregar en caso que se considere, según lo requiera la solución implementada.

- Todos los endpoints deben manejar los errores siguiendo el estándar HTTP, devolviendo el código de estado que corresponda a cada situación (por ejemplo 400 para datos inválidos, 401 para falta de autenticación o token inválido, 403 para falta de permisos, 404 para recursos inexistentes, etc) junto con el detalle del error en el cuerpo de la respuesta.
- Salvo el login, todos los endpoints deben estar protegidos con autenticación: la ausencia o invalidez del token debe rechazarse con 401 antes de evaluar cualquier otra regla de negocio, y los permisos según el rol del usuario autenticado deben verificarse en el propio endpoint, no solo ocultarse en la interfaz.

Autenticación

- `POST /api/auth/login` — iniciar sesión.
- `POST /api/auth/logout` — cerrar sesión.
- `GET /api/auth/me` — obtener los datos del usuario autenticado.
  - **Rol(es)**: cualquier usuario autenticado de las tres interfaces operativas.
  - **Entrada**: token de acceso en el header Authorization (Bearer).
  - **Salida**: id, nombre, apellido, email, rol, sector asociado (si corresponde).
  - **Reglas** de negocio: validar que el token sea válido y no haya expirado.
  - **Errores**: 401 si falta el token, es inválido o expiró.

Sectores y datos generales

- `GET /api/sectors – listar sectores.`
  - **Rol(es)**: Servicio de Alimentación, Sector Comedor. El Jefe de Servicio no lo necesita: opera únicamente sobre su propio sector, ya disponible mediante `GET /api/auth/me`.
  - **Entrada**: parámetro de query opcional `estado` (`activo` por defecto; `todos` para incluir inactivos, necesario al consultar períodos históricos).
  - **Salida**: listado de sectores (id, nombre, descripción, estado).
  - **Reglas de negocio**: por defecto solo deben listarse sectores activos, para no ofrecerlos como opción en nuevos cupos o autorizaciones.
  - **Errores**: 403 si lo invoca un usuario con rol Jefe de Servicio.

- `GET /api/sectors/{sector_id}` – obtener un sector.
  - **Rol(es)**: Servicio de Alimentación, Sector Comedor.
  - **Entrada**: `sector_id` en la URL.
  - **Salida**: datos completos del sector, incluyendo inactivos.
  - **Reglas de negocio**: sin restricción por estado — un sector desactivado debe poder consultarse para mantener trazabilidad histórica; solo se restringe su uso para nuevas asociaciones.
  - **Errores**: 404 si el sector no existe.
- `GET /api/menu-types` – listar tipos de menú.
  - **Rol(es)**: cualquier usuario autenticado de las tres interfaces operativas (Jefe de Servicio, Servicio de Alimentación, Sector Comedor); se usa para poblar los combos de selección de menú y funcionalidades que lo requieran .
  - **Entrada**: ninguna.
  - **Salida**: listado completo de tipos de menú predefinidos.
  - **Reglas de negocio**: ninguna; catálogo fijo, sin filtro de activo/inactivo previsto en esta etapa.
  - **Errores**: —
- `GET /api/personnel-types` – listar tipos de personal.
  - **Rol(es)**: cualquier usuario autenticado de las tres interfaces operativas.
  - **Entrada**: ninguna.
  - **Salida**: listado completo de tipos de personal predefinidos.
  - **Reglas de negocio**: ninguna.
  - **Errores**: —
- `GET /api/configuration` – consultar parámetros generales necesarios para las interfaces.
  - **Rol(es)**: cualquier usuario autenticado.
  - **Entrada**: ninguna.
  - **Salida**:
  - **Reglas de negocio**: pendiente de cerrar la definición de "Ajustes del sistema".
  - **Errores**: —

Personal

- `GET /api/personnel` — buscar y listar personal.
  - **Rol(es)**: Jefe de Servicio (limitado a su propio sector y del listado de residentes, ver Reglas de negocio), Servicio de Alimentación, Sector Comedor.

- **Entrada**: filtros por query — DNI, nombre o apellido, sector, tipo de personal, estado; paginación (máximo 25 por página).
- **Salida**: listado paginado de personas con sus datos básicos y el sector.
- **Reglas de negocio**: un Jefe de Servicio solo puede ver personal con asignación vigente en su propio sector y, adicionalmente, buscar personas de tipo Residente, sin importar qué sector pida por parámetro — el backend debe forzarlo según el usuario autenticado, no confiar en lo que envíe el cliente; Servicio de Alimentación y Sector Comedor pueden buscar en todos los sectores; por defecto listar solo personas activas.
- **Errores**: 400 si los filtros son inválidos.

- `GET /api/personnel/{person_id}` — obtener una persona.
  - **Rol(es)**: Jefe de Servicio (solo si la persona tiene asignación vigente en su sector o si es de Tipo residente), Servicio de Alimentación, Sector Comedor.
  - **Entrada**: person_id en la URL.
  - Salida: datos completos de la persona, incluyendo el sector y el tipo de personal.
  - **Reglas de negocio**: si lo llama un Jefe de Servicio y la persona no tiene asignación vigente en su sector o no es residente, debe rechazarse aunque la persona exista, para no exponer datos de personal de otros sectores.
  - **Errores**: 404 si no existe; 403 si un Jefe de Servicio consulta a alguien fuera de su sector.
- `GET /api/sectors/{sector_id}/personnel` — listar el personal de un sector.
  - **Rol(es)**: Jefe de Servicio (solo su propio sector_id), Servicio de Alimentación, Sector Comedor.
  - **Entrada**: sector_id en la URL; parámetro de query opcional fecha (por defecto la fecha actual); paginación.
  - **Salida**: listado del personal activo perteneciente al sector.
  - **Reglas de negocio:** un Jefe de Servicio que pida un sector_id distinto al propio debe recibir 403.
  - **Errores**: 403 si un Jefe de Servicio pide un sector distinto al propio; 404 si el sector no existe.
- `POST /api/personnel – crear una persona.`
  - **Rol(es)**:Rol(es): Jefe de Servicio, Servicio de Alimentación.
  - **Entrada**: DNI, nombre, apellido, tipo de personal, tipo menu y sector cuando corresponda.

- **Salida**: datos de la persona creada.
- **Reglas de negocio**:
  - No debe existir otra persona con el mismo DNI
  - Un Jefe de Servicio solo podrá crear personas de tipo Personal Fijo asociadas a su propio sector. El sector se determinará a partir del usuario autenticado y no deberá confiarse en un `sector_id` enviado por el cliente.
  - Las personas de tipo Residente no tendrán sector asociado.
  - La creación o modificación de Residentes quedará a cargo del **Servicio de Alimentación**. El Jefe de Servicio solamente podrá buscarlos y agregarlos a sus pedidos.
  - El tipo de personal indicado deberá existir.
  - La nueva persona quedará activa por defecto.
- **Errores**: 400 si faltan datos obligatorios o alguno de los datos es inválido.; 409 si ya existe una persona con el mismo DNI ; 403 si un Jefe de Servicio intenta crear personal fuera de su sector o una persona de tipo Residente

- `PUT /api/personnel/{person_id} – modificar una persona.`
  - **Rol(es)**:Rol(es): Jefe de Servicio, Servicio de Alimentación.
  - **Entrada**: `person_id` en la URL y los datos modificables de la persona.
  - **Salida**: datos actualizados de la persona.
  - **Reglas de negocio**:
    - Un Jefe de Servicio solo podrá modificar personas de tipo Personal Fijo pertenecientes a su propio sector.
    - Un Jefe de Servicio no podrá modificar Residentes ni trasladar una persona a otro sector.
    - El Servicio de Alimentación podrá modificar Personal Fijo y Residentes.
    - No deberá permitirse modificar el DNI de forma que se genere una duplicación con otra persona registrada.
    - Si se modifica el tipo de personal de Personal Fijo a Residente, deberá eliminarse su asociación con el sector.
    - Si una persona pasa de Residente a Personal Fijo, deberá indicarse obligatoriamente el sector al que pertenece.

    - Las modificaciones de los datos de una persona no deberán alterar pedidos, autorizaciones o entregas registrados anteriormente.
  - **Errores**: 404 si la persona no existe; 400 si los datos ingresados son inválidos; 403 si un Jefe de Servicio intenta modificar una persona que no pertenece a su sector o una persona de tipo Residente; 409 si el DNI ingresado corresponde a otra persona.

- `DELETE /api/personnel/{person_id} – eliminar o desactivar una persona.`
  - **Rol(es)**:Rol(es): Jefe de Servicio, Servicio de Alimentación.
  - **Entrada**: `person_id` en la URL.
  - **Salida**: confirmación de la operación realizada.
  - **Reglas de negocio:**
    - Un Jefe de Servicio solo podrá realizar la operación sobre Personal Fijo perteneciente a su propio sector.
    - Si la persona no tiene pedidos, autorizaciones, entregas u otros registros históricos asociados, podrá eliminarse.
    - Si existen registros históricos asociados, la persona no deberá eliminarse físicamente y deberá marcarse como inactiva.
    - Una persona inactiva no podrá incorporarse a nuevos pedidos, pero deberá continuar disponible para consultas históricas.
    - Los pedidos, autorizaciones y entregas registrados previamente no deberán modificarse como consecuencia de esta operación.
  - **Errores**: 404 si la persona no existe; 403 si un Jefe de Servicio intenta modificar una persona que no pertenece a su sector o una persona de tipo Residente;

Pedidos de comidas

`GET /api/requests – listar pedidos.`

- Rol(es): Jefe de Servicio.

- Entrada: filtros por query — rango de fechas, tipo de comida, tipo de menú, estado (confirmado/autorizado/rechazado); paginación.
- Salida: listado paginado de pedidos con sus totales por comida y tipo de menú.
- Reglas de negocio: el sector se determina por el usuario autenticado, nunca por un parámetro que mande el cliente — un Jefe de Servicio solo ve los pedidos de su propio sector, y los ve junto con el resto de los Jefes de ese sector.
- Errores: —

`POST /api/requests` — crear un pedido.

- Rol(es): Jefe de Servicio.
- Entrada: periodicidad (diaria, semanal o mensual), fecha o rango de fechas, y el detalle por comida (desayuno/almuerzo/merienda/cena): la lista de personas (person_id) incluidas. El sector se toma del usuario autenticado, no se recibe del cliente.
- Reglas de negocio: cada persona de tipo Personal fijo incluida debe tener como sector el del Jefe de Servicio que realiza la carga; los Residentes pueden incluirse sin esa validación; no debe permitirse cargar personas inactivas; las personas se cargan dentro del cupo asignado al sector para esa comida hasta agotarlo, y las que excedan ese límite quedan marcadas como excepción. No deberá registrarse más de una vez una misma solicitud para la combinación persona, sector, fecha y tipo de comida.
- Errores: 400 si faltan datos o una persona no tiene asignación vigente en la fecha de la comida; 409 si ya existe un pedido para el mismo sector y fecha

`GET /api/requests/{request_id}` — obtener un pedido.

- Rol(es): Jefe de Servicio, solo si el pedido pertenece a su sector.
- Entrada: request_id en la URL.
- Salida: detalle completo del pedido — cantidades pedidas por comida y tipo de menú, personas incluidas, estado; si ya fue revisado por Servicio de Alimentación, incluir también las cantidades autorizadas como referencia de solo lectura, sin mezclarlas con las pedidas.

- Reglas de negocio: —
- Errores: 404; 403 si el pedido no pertenece al sector del Jefe de Servicio que consulta.

`PUT /api/requests/{request_id} – modificar un pedido.`

- Rol(es): Jefe de Servicio, solo del sector propio.
- Entrada: los mismos campos que la creación.
- Salida: el pedido actualizado.
- Reglas de negocio: solo podrá modificarse un pedido en estado confirmado. Al modificar las personas incluidas, el sistema deberá recalcular la condición Regular o Excepción de las solicitudes según el cupo actual del sector. Un pedido autorizado o rechazado no podrá ser modificado por el Jefe de Servicio.
- Errores: 404 si el pedido no existe; 403 si no pertenece al sector del usuario; 409 si el pedido ya fue autorizado o rechazado.

`DELETE /api/requests/{request_id} – eliminar un pedido`

- Rol(es): Jefe de Servicio, sector propio.
- Entrada: request_id en la URL.
- Salida: confirmación vacía.
- Reglas de negocio: solo podrá eliminarse un pedido en estado confirmado, mientras no haya sido autorizado o rechazado por el Servicio de Alimentación.
- Errores: 404 si el pedido no existe; 403 si no pertenece al sector del usuario; 409 si el pedido ya fue autorizado o rechazado.

Autorización de pedidos

`GET /api/authorizations – listar pedidos pendientes o autorizados.`

- Rol(es): Servicio de Alimentación.

- Entrada: filtros por query — rango de fechas, sector, tipo de comida, tipo de menú, estado (confirmado/autorizado/rechazado); paginación. Etapa 2 pide vistas diarias y semanales, y consultar únicamente los períodos que tengan pedidos registrados, así que el filtro por fecha debería poder devolver solo fechas con datos en vez de un calendario vacío.
- Salida: listado paginado de pedidos confirmados por los Jefes de Servicio, con las cantidades pedidas y, si ya se ajustaron, las autorizadas, siempre por separado.
- Reglas de negocio: los pedidos en estado confirmado se consideran pendientes de autorización. Los totales deberán calcularse a partir de las solicitudes nominadas.
- Errores: —

`GET /api/authorizations/{request_id} – obtener el detalle de un pedido para su autorización.`

- Rol(es): Servicio de Alimentación.
- Entrada: request_id en la URL.
- Salida: detalle completo discriminado por comida y tipo de menú, con cantidades pedidas y autorizadas por separado, y las personas incluidas.
- Reglas de negocio: —
- Errores: 404 si el pedido no existe.

`PUT /api/authorizations/{request_id} – autorizar un pedido`

- Rol(es): Servicio de Alimentación.
- Entrada: listado de las solicitudes nominadas del pedido indicando cuáles quedan autorizadas y cuáles no autorizadas.
- Salida: pedido actualizado con las personas y comidas autorizadas, junto con los totales resultantes.
- Reglas de negocio: solo podrán autorizarse solicitudes que formen parte del pedido original. La autorización deberá realizarse de manera independiente para cada persona y tipo de comida. La interfaz podrá permitir la selección individual o masiva de solicitudes, manteniendo en todos los casos el registro nominado de cada autorización. Las

cantidades autorizadas se calcularán automáticamente a partir de las solicitudes autorizadas.
- Errores: 404 si el pedido no existe; 409 si el pedido ya está autorizado o rechazado; 400 si se intenta autorizar una solicitud que no pertenece al pedido.

`POST /api/authorizations/{request_id}/reject – rechazar un pedido.`

- Rol(es): Servicio de Alimentación.
- Entrada: request_id en la URL; posiblemente un motivo de rechazo, para trazabilidad (no está pedido en el documento pero es un dato barato de guardar).
- Salida: el pedido en estado rechazado.
- Reglas de negocio: un pedido rechazado no debe quedar disponible para entrega, pero sí visible en el historial.
- Errores: 409 si ya estaba autorizado.

Cupos por sector

`GET /api/quotas – listar cupos.`

- Rol(es): Servicio de Alimentación (gestión); Jefe de Servicio, solo lectura del cupo de su propio sector (Etapa 2 pide "visualizar el cupo asignado a su sector... mientras se realiza la carga"); Sector Comedor, para consultas en la interfaz.
- Entrada: filtros por query — sector, tipo de comida; paginación.
- Salida: listado de cupos (sector, comida, cantidad).
- Reglas de negocio: un Jefe de Servicio solo puede ver los cupos de su propio sector, igual que con /api/sectors.
- Errores: 403 si un Jefe de Servicio pide cupos de otro sector.

`POST /api/quotas – crear un cupo.`

- Rol(es): Servicio de Alimentación.
- Entrada: sector, comida, cantidad
- Salida: el cupo creado.

- Reglas de negocio: no debe poder crearse un cupo para un sector inactivo, igual que el resto de las asociaciones del sistema con sectores.
- Errores: 400 si el sector está inactivo o los datos son inválidos.

`GET /api/quotas/{quota_id} — obtener un cupo.`

- Rol(es): Servicio de Alimentación; Jefe de Servicio, solo el propio; Sector Comedor.
- Entrada: quota_id en la URL.
- Salida: datos completos del cupo.
- Reglas de negocio: mismo scoping por sector que en el listado para el Jefe de Servicio.
- Errores: 404; 403 según corresponda.

`PUT /api/quotas/{quota_id} — modificar un cupo.`

- Rol(es): Servicio de Alimentación.
- Entrada: los mismos campos que la creación.
- Salida: el cupo actualizado.
- Reglas de negocio: la modificación de un cupo se aplicará únicamente a los pedidos que se registren con posterioridad al cambio. No deberá modificar la condición Regular o Excepción de las solicitudes pertenecientes a pedidos ya registrados. No podrá existir más de un cupo para la misma combinación de sector y tipo de comida.
- ~~—~~**Errores: 400 si la nueva cantidad es menor** ~~a lo ya entregado en el período.~~ **a la cantidad ya autorizada o entregada para ese sector, fecha y tipo de comida.**

Consulta de comidas autorizadas

`GET /api/authorized-meals — consultar comidas autorizadas por fecha, DNI, sector o tipo de comida.`

- Rol(es): Sector Comedor (uso principal, para verificar autorizaciones al momento de la entrega); Servicio de Alimentación.
- Entrada: filtros por query — fecha, DNI,NyA, sector, tipo de comida.

- Salida: listado de personas autorizadas para esa combinación, con el tipo de menú autorizado y si ya fue entregada esa comida o no, para evitar entregas duplicadas.
- Reglas de negocio: solo deben aparecer pedidos en estado autorizado; debe indicarse cuánto de lo autorizado ya fue entregado.
- Errores: 400 si no se envía ningún filtro, para evitar traer todo el sistema de una.

`GET /api/authorized-meals/{person_id} – consultar las comidas autorizadas para una persona.`

- Rol(es): Sector Comedor, Servicio de Alimentación.
- Entrada: person_id en la URL; parámetro de query fecha (por defecto la fecha actual).
- Salida: las comidas autorizadas para esa persona en esa fecha (desayuno/almuerzo/merienda/cena, tipo de menú, si ya fue entregada).
- Reglas de negocio: igual que el endpoint anterior.
- Errores: 404 si la persona no existe.

Entregas

`GET /api/deliveries – listar entregas.`

- Rol(es): Sector Comedor (las del turno en curso o consultas propias); Servicio de Alimentación (para reportes).
- Entrada: filtros por query — fecha, sector, tipo de comida; paginación.
- Salida: responsable y listado de entregas registradas.
- Reglas de negocio: —
- Errores: —

`POST /api/deliveries – registrar una entrega.`

- Rol(es): Sector Comedor.
- Entrada: personas incluidas en el retiro (person_id de cada una), tipo de comida, tipo de menú entregado, y quién es el responsable del retiro (puede ser una de las personas incluidas o alguien que retira en representación de otras).

- Salida: la entrega registrada, con fecha y hora, usuario del Sector Comedor que la efectuó, personas incluidas y responsable del retiro.
- Reglas de negocio: cada persona incluida debe tener autorización vigente para esa comida y fecha que todavía no haya sido entregada, para no permitir retirar dos veces lo mismo; fecha, hora y usuario se completan automáticamente en el servidor, nunca los envía el cliente. Debe permitirse registrar la entrega de comidas correspondientes a una o varias personas en una misma operación
- 409 si alguna persona ya tiene esa comida entregada; 400 si alguna persona no tiene autorización vigente para esa comida y fecha.

`GET /api/deliveries/{delivery_id} – obtener el detalle de una entrega.`

- Rol(es): Sector Comedor, Servicio de Alimentación.
- Entrada: delivery_id en la URL.
- Salida: detalle completo de la entrega.
- Reglas de negocio: —
- Errores: 404.

`GET /api/deliveries/summary – obtener el resumen de entregas de una fecha o tipo de comida.`

- Rol(es): Sector Comedor (para ver el total autorizado y entregado durante el día, según Etapa 2); Servicio de Alimentación.
- Entrada: fecha o tipo de comida; sector opcional.
- Salida: totales autorizados vs. entregados, discriminados por comida y tipo de menú.
- Reglas de negocio: —
- Errores: —

Resúmenes

`GET /api/summaries/requests – obtener el resumen de pedidos.`

- Rol(es): Servicio de Alimentación; Jefe de Servicio, limitado a su propio sector.

- Entrada: filtros por query — rango de fechas, sector, tipo de comida, tipo de menú, agrupamiento (día/semana/mes, sector, comida, tipo de menú).
- Salida: totales de comidas pedidas, agrupados según el filtro.
- Reglas de negocio: si lo llama un Jefe de Servicio, se limita a su propio sector.
- Errores: —

`GET /api/summaries/authorizations – obtener el resumen de autorizaciones.`

- Rol(es): Servicio de Alimentación.
- Entrada: los mismos filtros que el resumen de pedidos.
- Salida: totales pedidos vs. autorizados, con la diferencia entre ambos.
- Reglas de negocio: —
- Errores: —

`GET /api/summaries/deliveries – obtener el resumen de entregas.`

- Rol(es): Servicio de Alimentación.
- Entrada: los mismos filtros que el resumen de pedidos.
- Salida: totales autorizados vs. entregados, con la diferencia entre ambos.
- Reglas de negocio: —
- Errores: —

Requisitos técnicos y pautas de trabajo

1 Requisitos técnicos

- El prototipo debe desarrollarse utilizando **Python**, **JavaScript**, **HTML5**, **CSS3** y **PostgreSQL**, respetando el modelo en capas **MVC**.
- Seguir las guías de estilo de Python y documentar todo el código Python con **docstrings**.
- Uso obligatorio de **Jinja** como motor de plantillas para la aplicación privada.
- La aplicación privada debe desarrollarse con el framework **Flask**.
- Validar todos los datos de entrada tanto en el cliente como en el servidor.
    - Las validaciones del lado del servidor deben implementarse en un módulo aparte que reciba los datos y devuelva el resultado.
    - Ejemplos de validaciones: solo números en un DNI, solo caracteres válidos para un nombre, opciones de lista que pertenezcan al conjunto permitido, etc.
- **Contraseñas**: deben almacenarse utilizando hash seguro (bcrypt, Argon2 o equivalente).
  Seguridad contra inyección SQL: toda interacción con la base de datos debe realizarse a través de **SQLAlchemy** para evitar inyecciones SQL.
- Se permite el uso de librerías que facilitan tareas (conexiones a servicios externos, parseo, patrones de buenas prácticas, validaciones de datos, etc.), siempre que todos los miembros del equipo puedan explicar su funcionamiento y propósito. Consultar con el ayudante si el uso de la misma está permitido.
- **Login:** no se permite el uso de librerías que abstraiga completamente el proceso (por ejemplo, Flask-Login), para asegurar comprensión de la implementación.
- Para la interacción con la base de datos, utilizar **SQLAlchemy** como ORM para mantener una capa de abstracción.
- Aplicar buenas prácticas de **semántica web en HTML5**, usando correctamente las etiquetas del lenguaje.
- Versionar todas las entregas con **git**, siguiendo **Versionado Semántico** (por ejemplo, etapa 1 → v1.x.x).
- Se permite el uso de frameworks CSS como **Bootstrap**, **Foundation**, **Bulma** o **Tailwind CSS**.
- **Manejo de logs**: implementar registro básico de eventos y errores en backend para facilitar depuración.

- **Control de dependencias**: mantener actualizados y controlados pyproject.toml (Poetry) y package.json con versiones definidas para evitar incompatibilidades.
- Todas las vistas deben ser **responsive** contemplando al menos tres resoluciones cómo mínimo (si quiere utilizar las sugeridas por el framework que utilice comuníquelo al ayudante):
    - < 767px (móviles)
    - 768px – 1024px (tablets)
    - > 1025px (portátiles y escritorio)

2 Pautas de trabajo y entrega

- La entrega es obligatoria. Todos los integrantes deben participar en la defensa.
- El ayudante evaluará el progreso y participación mediante consultas online y seguimiento en **GitLab**.
- El proyecto se deberá realizar en grupos de **cinco integrantes**.
- Deben ser visibles los aportes de cada integrante tanto en **GitLab** como en la defensa.
- Si bien las tareas deberán dividirse entre los integrantes para organizar el trabajo, todos los miembros del equipo deberán comprender el funcionamiento integral del sistema y ser capaces de responder preguntas sobre cualquiera de sus componentes.
- La defensa será virtual, salvo que se acuerde presencial por falta de medios técnicos (**micrófono y cámara**).
- El trabajo será evaluado desde el **servidor de la cátedra**. No se aceptarán entregas fuera del tiempo y forma establecidos.
- Funcionalidades no terminadas en la etapa 1 podrán completarse en la etapa siguiente.

3 Información del servidor

- **Lenguaje:** Python 3.14.4
- **Servidor web:** nginx/1.28.3 (Ubuntu)
- **Dependencias Python:** Poetry 2.4.1
- **Base de datos:** PostgreSQL 16